Principles

HOW THE DESIGN SYSTEM OPERATES

Speed over friction

Design systems exist to help teams move faster and should never be a gating function.

- **Default to existing components** and patterns before designing something new.
- Evolve existing patterns instead of duplicating them when small changes are needed.
- Teams **should not wait on system updates to ship critical work**.
- Divergence is allowed when needed, as long as it is intentional and documented.

Design-led system ownership

The system is design-led by necessity in the absence of a platform team.

- **Design leads system** stewardship, quality, and consistency across products.
- System updates should be documented and communicated clearly.
- Designers **work with cross-functional teams** to collaborate on shared patterns and processes for system updates.
- Governance relies on clarity, trust, and **shared ownership, not enforcement**.

Shared constraints, shared patterns

Like products and shared patterns should align by default.

- Products solving **similar problems should share patterns** where possible.
- Examples include checking, PCL, or high-volume VQ workflows like HI and PL.
- Shared surfaces such as directory, profile, settings, and docs should follow system constraints.
- **Alignment should happen early** in design, not during QA or handoff.

Flexibility where it matters

Autonomy increases as experiences become more specialized.

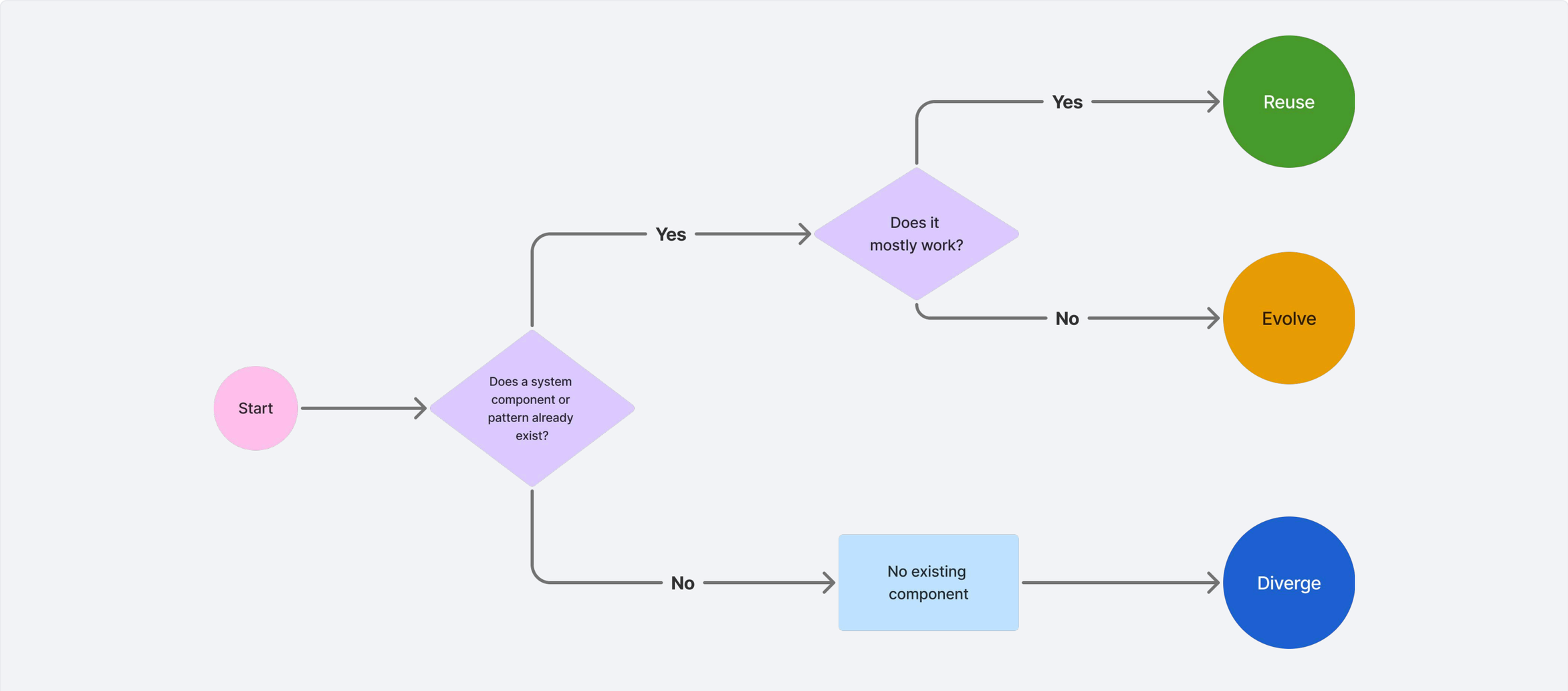
- Product-specific surfaces should be optimized for their users, **not over-standardized or forced into patterns** that don't work.
- The system should **enable creativity and innovation**, not restrict it
- The system should support variation without compromising usability or clarity.
- **Foundations still apply everywhere** like spacing, typography, and accessibility.

Decision Framework

HOW DO WE EVALUATE WHAT BELONGS IN THE SYSTEM VS WHAT DOESN'T?

🔁 Reuse | 🧩Evolve | 🌿Diverge

Question	Observation	Decision	Outcome
Does a system component or pattern already exist?	Looks the same and behaves the same	Reuse	Use **as-is**. Optimize for speed and consistency.
Does the pattern mostly work but needs changes?	Mostly the same with broader potential	Evolve	Extend or refine while keeping core structure.
Does the pattern fail a real user or business need?	Fundamentally different with clear justification	Diverge	Create a product-specific solution with intent.



Guardrails (Apply to All Paths)

Before deciding, teams should ask:

- Is this a shared surface or a product-specific feature?
- Does this affect multiple products or only one?
- Will this likely be reused in the future?
- Does this improve the customer experience in a meaningful way?

Shared Features Rule

For shared features (directory, profile, settings, etc.):

- Default: Reuse or evolve
- Diverge only if: There is a strong customer or business need that the system does not currently support

Special Cases

Highly variable components (e.g. transaction rows)

- Prefer guidelines over strict components.
- Allow product-level optimization within documented constraints.
- Build component with scalability in mind (e.g. edge cases, localization, responsiveness)

Time-sensitive work

- Ship what's needed. Document later.
- The system should learn from reality, not block delivery.

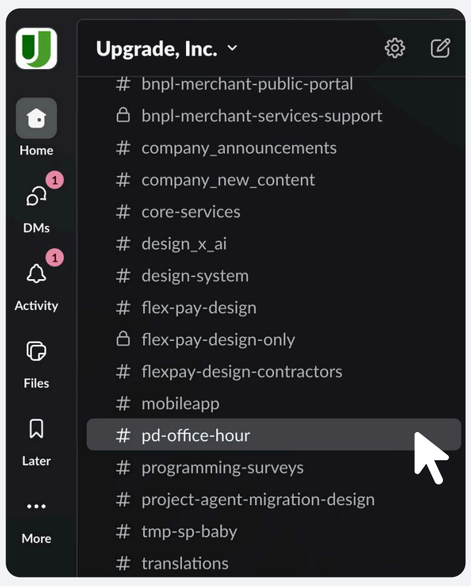
Contribution Model

HOW DO WE CONTRIBUTE NEW COMPONENTS AND PATTERNS?

Step 1: Post in Slack (pd-office-hour)

Designer must include:

- Problem (1 sentence)
- Why existing component doesn't work (1 sentence)
- Proposed solution (Figma link + max 2 screenshots)
- Reuse potential (where else this applies)



Step 2: Owner Decision (Within 48 hours)

Platform Owner:

- Responsive Web → **Steph**
- Mobile App → **Gabriel**

Possible outcomes:

- ✅ Approved
- ✏️ Approved with tweaks
- 🔁 Use existing pattern (owner to recommend)
- ❌ Product-only

If unclear schedule 15-min max meeting with both owners

Platform Owners

Steph Spicer
Responsive Web

Gabriel Keith
Mobile App

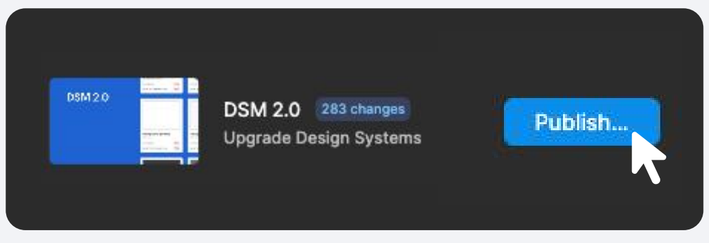
Step 3: Build + Publish

Designer:

- Add states + variants
- Follow tokens + accessibility
- Create or update documentation (support available via LLM)

Owner:

- Final check
- Publish to library
- Post "New in DS" update
- Add to release notes



Question	Observation	Decision	Outcome
Does a system component or pattern already exist?	Looks the same and behaves the same	Reuse	Use **as-is**. Optimize for speed and consistency.
Does the pattern mostly work but needs changes?	Mostly the same with broader potential	Evolve	Extend or refine while keeping core structure.
Does the pattern fail a real user or business need?	Fundamentally different with clear justification	Diverge	Create a product-specific solution with intent.